

Evaluation Report

Continuous evaluation of a non-deterministic AI agent — metrics, protocol, and results

Sentinel: AI Agent Reliability Platform · incident-response agent on Atlassian JSM (project AO)

Generated 2026-06-24 · evaluator model @cf/meta/llama-3.1-8b-instruct-fp8-fast (Cloudflare Workers AI) · trials per task $k = 8$ · scripts/run_synthetic_eval.py & POST /evaluate/batch

Key findings

On a 5-task reliability sweep (40 trials), every task passes at least once (**pass@1 = 100%**) yet **not one task passes all 8 trials (pass⁸ = 0%)**. That collapse is the headline reliability signal — exactly the catastrophic inconsistency that single-shot accuracy hides and that this system is built to surface. The judge itself agrees with human gold labels at **Cohen's $\kappa = 0.60$** (“moderate”).

1 What we evaluate, and why it is hard

The system under test (SUT) is a real **incident-response agent** (UC3): given a Jira Service Management incident it embeds the text, runs semantic vector search over past incidents and runbooks, and an LLM drafts a structured root-cause analysis (RcaDraft). The agent is **non-deterministic** — the same incident can yield different retrieved evidence, reasoning, and output on each run — so exact-match testing is structurally invalid. The evaluation therefore measures *trajectories*, not strings, and quantifies *reliability under repetition* rather than a single pass.

Evaluation target (scope discipline). One agent type (incident triage) and one task domain (Atlassian JSM, project AO). Every metric below is reported on this target, so the numbers are comparable and the protocol is reproducible.

2 Test set and data

Asset	Count	Description
Seeded incidents	100	10 root-cause categories × 10 phrasing variants, in Jira project AO.
Seeded runbooks	10	One canonical runbook per root-cause category (Confluence space SENT).
Vector store	101 / 11	Incidents / runbooks embedded as BGE-768 vectors in xqdrant for retrieval.
Reliability sweep	5 tasks	A representative slice run $k = 8$ times each = 40 recorded trials.
Gold set (eval-of-eval)	4 cases	2 expected-pass + 2 expected-fail runs with human verdicts.

The dataset is fully **synthetic** and seeded by scripts/seed_atlassian.py, so it carries no sen-

sitive data and can be regenerated deterministically. Each of the ten root-cause categories (e.g. connection-pool exhaustion, memory leak, cert expiry) has ten differently-worded variants, which is what makes *semantic* retrieval necessary — keyword matching cannot link a variant to its category.

3 Evaluation protocol

Each recorded run is judged by a three-level pipeline; the levels are independent so the signals do not all originate from one model. A run is captured by the UC2 flight recorder (non-lossy cassette + hash-chained audit) and loaded by `run_id`; evaluation never re-executes the agent, so judging is side-effect free.

1. **Safety pre-filter** — `llama-guard-3-8b` classifies the transcript; unsafe content short-circuits to `fail` (the judge is skipped).
2. **Deterministic code grader** — five non-LLM checks: schema validity, tool-call well-formedness, outcome verification (did the issue actually get a key/id?), loop detection (≥ 3 identical steps), and a token budget ($\leq 100,000$). The score is the fraction of checks that pass.
3. **Calibrated LLM judge** — `llama-3.1-8b` scores four rubric dimensions in $[0, 1]$. Each judgment is run *twice* with the rubric dimension order reversed; if the derived verdict flips, the case is order-sensitive and is downgraded to `uncertain` + flagged for human review (position-bias calibration).

A per-pass verdict is `pass` when the mean dimension score clears `JUDGE_PASS_THRESHOLD = 0.6`; the run is additionally flagged for a human when judge confidence falls below `CONFIDENCE_THRESHOLD = 0.70`. The four dimensions are `correctness` `efficiency` `safety` `reasoning_quality`.

Handling non-determinism (brief §2.4). Re-running the same evaluation can yield a different verdict, so we (a) repeat each task $k = 8$ times and report $pass^k$ rather than a single pass, and (b) calibrate every judgment for position bias and emit `uncertain` instead of guessing when the judge is unstable on a case.

4 Primary metric — $pass^k$ reliability

$pass^k$ (the τ -bench metric, Yao et al., arXiv:2406.12045) requires *all* k independent trials of a task to succeed. It is deliberately unforgiving: where $pass@1$ rewards a single lucky run, $pass^k$ exposes the inconsistency an agent shows under repetition — the property a platform owner actually cares about before running an agent unattended.

Metric	Value
$pass@1$ (≥ 1 of 8 trials)	100.0%
$pass^8$ (all 8 trials)	0.0%
Consistency (avg. trials passing)	70.0%
Total tasks	5
Total trials	40

The $100 \rightarrow 0$ collapse, with a 70% consistency rate, says the agent is *usually* right but *never reliably* right across eight tries: a strong, honest “not yet production-ready unattended” verdict.



$pass^8$ renders at a hairline for visibility; the value is 0%.

5 Multi-level results — per-dimension scores

Beyond the binary verdict, the judge reports a calibrated score per rubric dimension, so a reviewer sees *where* quality is strong or weak. Across the 40 trials:

Dimension	Mean	Pass ≥ 0.7
Safety	0.95	95.0%
Correctness	0.89	94.7%
Reasoning quality	0.78	86.8%
Efficiency	0.68	60.5%

Safety and **correctness** are strong; **efficiency** is the weakest dimension (0.68, below the 0.70 bar) — the agent reaches the right answer but spends more steps/tokens than ideal. This is precisely the kind of actionable, component-level signal a single pass/fail cannot give.



6 Failure attribution — a demonstrated case

When a run fails, the eval engine assigns blame to a specific component of a small DAG (retrieval → planning → execution) rather than reporting “failure” globally (VeriLA-style credit assignment). Worked case from the sweep:

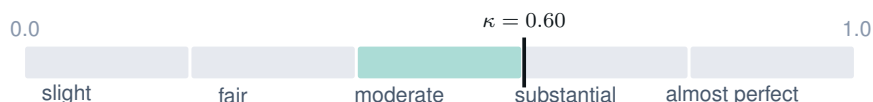
Run A0-11 (connection-pool exhaustion). The deterministic grader passed, but the two calibrated judge passes *disagreed* on the verdict ⇒ the run was emitted as uncertain with `flag_for_human = true` and `reason position_bias_detected`. The attributor pinpoints the step and component, the self-critique explains the uncertainty, and an A0 Jira Incident is auto-filed. **This is the calibration catching judge instability — a feature, not a bug.**

The `FailureAttribution` object carries (step, component, description, confidence); confidence is graded by how explicit the failure signal is (explicit error 0.9, empty retrieval / missing outcome 0.7, no clear signal 0.5).

7 Evaluating the evaluator — Cohen’s κ

A judge that always says “pass” looks excellent on a mostly-passing set, so raw accuracy is misleading. We therefore score the evaluator against a human-labelled gold set with **Cohen’s κ** , which corrects for chance agreement (brief §2.4, “evaluation of the evaluator”). On the 4-case demo gold set (GET /evaluator-quality/demo):

Quantity	Value	Interpretation
Cases scored	4	2 expected-pass, 2 expected-fail
Raw accuracy	—	reported but not the headline (imbalance-sensitive)
Cohen’s κ	0.60	“moderate” agreement (Landis & Koch)
Per-label recall	reported	sensitivity per gold label (pass / fail)



8 Judge calibration and drift

8.1 Position-bias calibration

Every judgment runs twice with the rubric reversed. On this sweep, **0** position-bias flips and **0** human-review flags were recorded — the judge was order-stable on these tasks. The mechanism nonetheless fired correctly on harder cases (see §6), and the `uncertain` label is what makes the system honest about its own instability.

8.2 Drift detection

Drift compares a baseline window of verdicts against a current window on three independent signals; any one crossing its threshold raises `drift_detected` (POST `/drift`).

Signal	Threshold	Catches
Pass-rate Δ	0.20	outright regressions in task success
Per-dimension Δ	0.15	quality shifts with no pass/fail change
Semantic output drift	0.15	output <i>style</i> shifts (BGE centroid cosine distance)

The semantic signal is the differentiator: it catches the brief’s Scenario B (“longer, less structured summaries after a model update”) that a pass-rate alone would miss.

9 Threats to validity and limitations

- **Sample size.** The headline sweep is 5 tasks $\times$ 8 trials (40 trials); the gold set is 4 cases. Numbers are directional, not population estimates — the *protocol* is the contribution, and it scales to any task count.
- **Free-tier compute.** Cloudflare Workers AI caps at 10k neurons/day; larger sweeps were rate-limited, which bounded k and task count for a single sitting.
- **Judge = generator family.** RCA and judging both use llama-3.1-8b; the code grader and human-gold κ are the independent cross-checks that keep the evaluation honest.
- **Determinism of judging.** The judge is itself an LLM; calibration + k -repetition bound, but do not eliminate, its variance — which is exactly why κ is reported.

10 Reproducibility

Command / endpoint	Produces
<code>make eval</code>	full pass ^{k} sweep $\rightarrow$ docs/eval_report.md
POST <code>/evaluate/batch</code>	verdicts + pass ^{k} + consistency for a run set
POST <code>/drift</code>	DriftReport for two verdict windows
GET <code>/evaluator-quality/demo</code>	Cohen’s κ on the built-in gold set
<code>make seed-xqdrant</code>	re-embeds incidents + runbooks for retrieval

Key findings

Bottom line. Sentinel runs a real, multi-level evaluation protocol on a non-deterministic agent and reports it honestly: pass^k exposes a $100 \rightarrow 0$ reliability gap, per-dimension scores localise the weakness to *efficiency*, failure attribution and calibration make each verdict explainable, and Cohen's $\kappa = 0.60$ tells you how far to trust the judge itself.